Challenge — Nim, il gioco dei fiammiferi

Lezione coperta: `L06` — cicli: `while`

Si svolge in aula al termine di `A02`. **Senza soluzioni** e **senza consegna**: per parlarne, la lezione dopo o il ricevimento.

Il gioco

Sul tavolo ci sono **21 fiammiferi**. Due giocatori, a turno, ne tolgono **1, 2 o 3**. **Perde chi prende l'ultimo.**

È un gioco senza fortuna: chi conosce il trucco vince sempre, o quasi. Le dieci challenge lo costruiscono un pezzo alla volta — una mossa, una partita, un computer che gioca a caso, mille partite — e poi cercano il trucco, prima nei numeri e poi nel ragionamento.

Cosa ti serve

la macchina	la VM del corso (`guida_VM.md`), o il tuo computer se preferisci
l'editor	Visual Studio Code, `guida_VSCode.md`
il repository	`~/InfoCourse`, aggiornato con `git pull` (`guida_GitHub.md`)
le tracce	la cartella `02_ProgrammareInPython/L06_challenge/`, dieci file da `ch_01.py` a `ch_10.py`

I numeri a caso. Dalla 3 in poi il computer gioca, e per scegliere una mossa a caso usa `numpy`. La riga è già scritta nella traccia:

```
mossa = np.random.randint(1, limite + 1)
```

estrae un intero **da 1 a `limite`**: il secondo estremo è escluso, per questo c'è il `+ 1`. Come funziona `import` lo vedremo in `L08`; per ora basta usarla.

Come si lavora

Copia la cartella delle tracce fuori dal repository, per esempio:

```
cp -r ~/InfoCourse/02_ProgrammareInPython/L06_challenge ~/challenge_L06
```

Se lavori dentro `~/InfoCourse`, al prossimo `git pull` le tue modifiche vanno in conflitto con le tracce.

Ogni traccia ha quattro parti: l'**enunciato** in testa, in commento; i **dati**, già pronti; il **codice**, che scrivi tu; le **risposte** alle domande, nei commenti in fondo. Dove ci sono più blocchi di dati, prova il programma su **tutti**, ricommentando il blocco attivo e togliendo il commento al successivo.

Alla fine `python3 ch_N.py`, da un terminale appena aperto, non deve dare errori. Le tracce 1, 2, 3 e 6 chiedono le mosse con `input()`: aspettano che tu le scriva.

Le challenge si concatenano: la 2 riparte dalla 1, la 3 dalla 2, la 4 dalla 3; la 6 usa quello che trovi nella 5; la 7 mette insieme la 4 e la 6, la 8 riparte dalla 7; la 9 e la 10 rifanno la 5 con regole diverse. Non avendo ancora le funzioni, «riusare» qui vuol dire ricopiare il tuo codice da un file all'altro.

Solo `while`, e niente `break`: un ciclo finisce quando la sua condizione diventa falsa. Se servono due motivi per finire, stanno tutti e due nella condizione.

Le dieci

#	Traccia	Difficoltà	Argomento
1	`ch_01.py` — Una mossa valida	•	il `while` che controlla un input
2	`ch_02.py` — Una partita in due	•	il turno che passa, il ciclo che finisce a zero
3	`ch_03.py` — Contro il computer	••	una mossa a caso, ma valida
4	`ch_04.py` — Mille partite a caso	••	un ciclo dentro un ciclo, frequenze e medie
5	`ch_05.py` — Chi vince da m fiammiferi	•••	ricordare gli ultimi tre esiti, senza liste
6	`ch_06.py` — La regola	••	dai numeri della 5 a una strategia
7	`ch_07.py` — Strategia contro caso	••	misurare una strategia
8	`ch_08.py` — Conviene cominciare?	•••	un risultato che sembra sbagliato
9	`ch_09.py` — Fino a k fiammiferi	•••	generalizzare, e dimostrare
10	`ch_10.py` — Vince chi prende l'ultimo	•••	la stessa idea con la regola rovesciata

La difficoltà sale **in fretta**. Le prime quattro si fanno con quello che hai visto a lezione e negli esercizi; dalla 5 in poi bisogna ragionare sul gioco più che su Python.

Riferimenti

`riepilogo_funzioni_python.md` per la sintassi, `L06_esercizi.md` per gli esempi svolti di contatori, accumulatori e cicli annidati, `L06_oca.md` per un altro gioco costruito un turno alla volta.